

Prompts para Base de datos, lógica, REST, fake Android, fake web y UX

Lucía Díaz Murcia

Índice

Prompts para Base de datos, lógica, REST, fake Android, fake web y UX	1
Prompt 1 — Diseño e implementación de la base de datos	3
Prompt 2 — Diseño e implementación de la lógica de negocio	4
Prompt 3 — Implementación del servidor REST	6
Prompt 4 — Lógica fake web	8
Prompt 5 — Lógica fake de la app Android	10
Prompt 6 — Diseño e implementación UX del navegador	12

Prompt 1 — Diseño e implementación de la base de datos

En cada fase, antes de modificar archivos inspecciona la estructura existente y conserva los archivos ajenos.

Todo fichero creado o modificado debe comenzar con una cabecera que incluya:

- nombre del fichero;
- autor: `Lucía Díaz Murcia`;
- descripción;
- fecha: `2026-09-25`;
- copyright: `Copyright (c) 2026`.

Cada función debe tener inmediatamente encima una cabecera de comentario con su diseño lógico en nuestra notación. Usa comentarios adicionales breves, únicamente si explican una decisión importante.

El diseño común obligatorio es:

MEDICIONES = [(id: N, tipo: Text, valor: R, fecha: DateTime)] (Para la base de datos)

(Para las funciones)

tipo: Text, valor: R --> guardarMediciones() -->

mediciones: [(id: N, tipo: Text, valor: R, fecha: DateTime)] <-- mostrarMediciones() <--

No crees una clase `Medición`. Los datos se pasan directamente como `tipo` y `valor`. El servidor crea `id` y `fecha`.

La base de datos debe contener por ahora una única tabla:

MEDICIONES = [(id: N, tipo: Text, valor: R, fecha: DateTime)]

Crea los ficheros SQL necesarios para:

1. Crear la estructura de la tabla `mediciones`.
2. Insertar datos de ejemplo.
3. Eliminar la estructura solo si es necesario para reiniciar el entorno de pruebas.

La tabla debe tener:

id: N identificador único y clave primaria y ser autoincremental

tipo: Text de momento los valores esperados: CO2 o TEMPERATURA

valor: R admite valores negativos, por ejemplo -12

fecha: DateTime generado por el servidor

Genera tests automáticos de base de datos en una base de datos exclusiva de pruebas. Los tests deben hacer exactamente este ciclo:

1. Vaciar las tablas de la base de datos de pruebas.
2. Comprobar que están vacías.
3. Insertar mediciones ficticias: al menos una de CO2 con valor 500 y una de TEMPERATURA con valor -19.
4. Comprobar número de filas, tipo, valor y fecha de cada medición.
5. Borrar las mediciones insertadas.
6. Comprobar de nuevo que la tabla queda vacía.

Nunca vacíes una base de datos de producción. Usa una configuración explícita de entorno de pruebas. Indica el comando exacto para crear la estructura y ejecutar los tests. Mientras lo haces ves preguntando las dudas que tengas durante el proceso o ves preguntándome que propuesta quiero elegir paso a paso.

Prompt 2 — Diseño e implementación de la lógica de negocio

Antes de modificar archivos inspecciona la estructura que hay y conserva los archivos que tengo. Mientras lo haces ves preguntando las dudas que tengas durante el proceso o ves preguntándome que propuesta quiero elegir paso a paso.

Todo fichero creado o modificado debe comenzar con una cabecera que incluya:

- nombre del fichero;
- autor: `Lucía Díaz Murcia`;
- descripción;
- fecha: `2026-09-25`;
- copyright: `Copyright (c) 2026`.

Cada función debe tener inmediatamente encima una cabecera de comentario con su diseño lógico en nuestra notación. Usa comentarios adicionales breves, únicamente si explican una decisión importante.

El diseño común obligatorio es:

```
MEDICIONES = [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]
```

```
tipo: Text, valor: R --> guardarMediciones() -->
```

```
mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]
```

```
<-- mostrarMediciones() <--
```

No crees una clase `Medicion`. Los datos se pasan directamente como `tipo` y `valor`. El servidor crea `id` y `fecha`.

Implementa la lógica de negocio PHP usando exactamente este **diseño de base de datos**:

`MEDICIONES = [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]`

También usa el diseño de la lógica de negocio:

```
----- LogicaNegocio -----
|
| mediciones:[(id:N, tipo:Text, valor:R ,fecha:DateTime) ]
|
tipo: Text, valor: R --> | guardarMediciones() -->
|
|
mediciones: [ (id: N, tipo: Text, |
valor: R, fecha: DateTime ) ] <- | mostrarMediciones() <--
|
-----
```

Genera tests automáticos de la lógica que usen la base de datos de pruebas. Antes de cada prueba, vacía la tabla de pruebas; inserta los datos ficticios necesarios; comprueba los resultados; y al terminar borra las mediciones insertadas y verifica que la tabla queda vacía.

Incluye cómo ejecutar los tests.

Prompt 3 — Implementación del servidor REST

Revisa los archivos existentes y crea nuevos si es necesario. Mientras lo haces ves preguntando las dudas que tengas durante el proceso o ves preguntándome que propuesta quiero elegir paso a paso.

Todo fichero creado o modificado debe comenzar con un comentario como cabecera que incluya:

- nombre del fichero;
- autor: `Lucía Díaz Murcia`;
- descripción;
- fecha: `2026-09-25`;
- copyright: `Copyright (c) 2026`.

Cada función debe tener encima un comentario con su diseño lógico en nuestra notación. Usa comentarios adicionales breves para explicar una decisión importante.

El diseño común obligatorio es:

```
MEDICIONES = [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]
```

```
tipo: Text, valor: R --> guardarMediciones() ->
```

```
mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime) ]<-- mostrarMediciones() <-
```

No crees una clase `Medicion`. Los datos se pasan directamente como `tipo` y `valor`. El servidor crea `id` y `fecha`.

Implementa el servidor REST PHP para XAMPP y Plesk usando exclusivamente estos diseños:

Rutas REST:

POST /mediciones

```
tipo: Text, valor: R --> guardarMediciones()
```

El JSON :{ "tipo": "CO2", "valor": 500 }

GET /mediciones

```
mostrarMediciones() -->mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime) ]
```

El JSON: [{ "id": 1, "tipo": "CO2", "valor": 500, "fecha": "2026-09-25T10:30:00" },
{ "id": 2, "tipo": "TEMPERATURA", "valor": -19, "fecha": "2026-09-25T10:31:00" }]

Y el diseño de la lógica:

```
----- LogicaNegocio -----  
|  
| mediciones:[(id:N, tipo:Text, valor:R, fecha:DateTime) ]  
|  
tipo: Text, valor: R --> | guardarMediciones() -->  
|  
|  
|  
mediciones: [ (id: N, tipo: Text, |  
valor: R, fecha: DateTime) ]<-| mostrarMediciones() <--  
|  
-----
```

El servidor debe delegar toda persistencia en la lógica de negocio y usar PDO con consultas preparadas.

Incluye configuración separada para desarrollo XAMPP, producción Plesk y pruebas. Nunca incluyas credenciales reales en el repositorio.

Genera tests automáticos de integración para `POST /mediciones` y `GET /mediciones`. Los tests deben usar exclusivamente la base de datos de pruebas y seguir este ciclo: vaciar, insertar datos ficticios, comprobar resultados, borrar las mediciones insertadas y comprobar que la tabla queda vacía.

Indica cómo arrancar el servidor local, cómo configurar Plesk y cómo ejecutar los tests.

Prompt 4 — Lógica fake web

Revisa cada archivo que tengo y crea los que sean necesarios para funcionar, pero asegurate de que todos tengan este encabezado:

- nombre del fichero;
- autor: `Lucía Díaz Murcia`;
- descripción;
- fecha: `2026-09-25`;
- copyright: `Copyright (c) 2026`.

Cada función debe tener inmediatamente encima un comentario con su diseño lógico en nuestra notación. Usa comentarios adicionales breves para explicar algo si es necesario. Mientras lo haces ves preguntando las dudas que tengas durante el proceso o ves preguntándome que propuesta quiero elegir paso a paso.

El diseño obligatorio es:

MEDICIONES = [(id: N, tipo: Text, valor: R, fecha: DateTime)]

mediciones: [(id: N, tipo: Text, valor: R, fecha: DateTime)]

<-- mostrarMediciones() <--

No crees una clase `Medicion`. Los datos se pasan directamente como `tipo` y `valor`. El servidor crea `id` y `fecha`.

Diseño REST obligatorio:

GET /mediciones

mostrarMediciones() --> [(id: N, tipo: Text, valor: R, fecha: DateTime)]

Diseño de la lógica de negocio:

```
----- lógica negocio -----  
|  
|  
tipo: Text, valor: R --> | guardarMediciones() --x  
|  
|  
mediciones: [ (id: N, tipo: Text, |  
valor: R, fecha: DateTime ) ] <--| mostrarMediciones() --x  
|  
-----
```

Implementa una lógica fake web usando el diseño de la lógica REST y las rutas indicado arriba.

Crea un nuevo archivo llamado PeticionarioREST de JavaScript con:

```
mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ] <-- pedirMediciones() --x
```

Debe poder llamar a `GET /mediciones` y devolver la lista recibida.

La URL REST debe estar configurada en un único sitio, fácil de cambiar entre XAMPP y Plesk.

Añade tests automáticos del peticionario web . Deben verificar el GET, el JSON generado y el manejo de un error de red o HTTP. Indica cómo ejecutar los tests.

Prompt 5 — Lógica fake de la app Android

Revisa cada archivo que tengo y crea los que sean necesarios para funcionar, pero asegurate de que todos tengan este encabezado:

- nombre del fichero;
- autor: `Lucía Díaz Murcia`;
- descripción;
- fecha: `2026-09-25`;
- copyright: `Copyright (c) 2026`.

Cada función debe tener inmediatamente encima un comentario con su diseño lógico en nuestra notación. Usa comentarios adicionales breves para explicar algo si es necesario.

El diseño obligatorio es:

MEDICIONES = [(id: N, tipo: Text, valor: R, fecha: DateTime)]

tipo: Text, valor: R --> guardarMediciones() -->

No crees una clase `Medicion`. Los datos se pasan directamente como `tipo` y `valor`. El servidor crea `id` y `fecha`.

Implementa una lógica fake en Android utilizando el diseño de la lógica REST y de las rutas.

```
----- LogicaNegocio -----
|
| mediciones:[(id:N, tipo:Text, valor:R, fecha:DateTime )]
|
|
| tipo: Text, valor: R --> | guardarMediciones() -->
|
|
|
mediciones: [ (id: N, tipo: Text, |
| valor: R, fecha: DateTime ) ]<-| mostrarMediciones() <--
|
|
|-----
```

POST /mediciones

tipo: Text, valor: R --> guardarMediciones()->

Completa `PeticionarioRest` con esta función estática:

tipo: Text, valor: R --> enviarMedicion() --x

La lógica fake debe poder enviar la medición del beacon.

Configura la URL base REST en un único archivo o constante, fácil de cambiar entre XAMPP y Plesk. No rompas el servicio BLE actual: la lógica fake debe coexistir con él y servir para probar el envío con beacon.

Añade tests automáticos con servidor HTTP simulado que comprueben:

1. La ruta usada es `/mediciones`.
2. El método es POST.
3. El JSON incluye solo `tipo` y `valor`.
4. Se procesa correctamente una respuesta 201.
5. Se informa correctamente de un error HTTP.

Indica cómo ejecutar los tests.

Prompt 6 — Diseño e implementación UX del navegador

Cada fichero debe tener cabecera con:

- nombre del archivo;
- autor `Lucía Díaz Murcia`;
- descripción;
- fecha `2026-09-25`;
- copyright `Copyright (c) 2026`.

Cada función debe incluir su cabecera de diseño lógico.

Ejemplos:

```
mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]  
    <-- mostrarMediciones() <--
```

Incluye tests automáticos de interfaz que comprueben:

1. Renderizado inicial.
2. Estado de carga.
3. Estado de error.
4. Renderizado de filas recibidas por GET.

Usa respuestas REST simuladas e indica cómo ejecutar los tests.

Teniendo en cuenta lo anterior diseña e implementa la experiencia de usuario del navegador para el proyecto web existente. Antes de modificar archivos, inspecciona su estructura y reutiliza sus convenciones. Mientras lo haces ves preguntando las dudas que tengas durante el proceso o ves preguntándome que propuesta quiero elegir paso a paso.

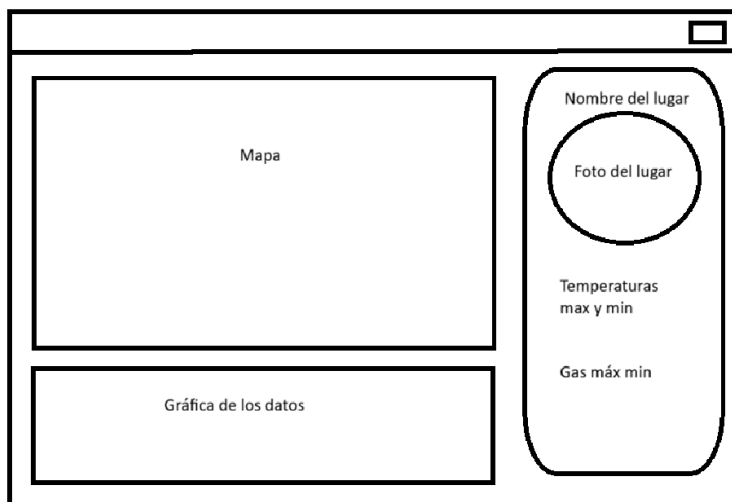
Usa la lógica fake web y estas ruta REST:

GET /mediciones

```
mostrarMediciones() --> [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]
```

```
----- PeticionarioREST -----  
|  
|  
mediciones: [ (id: N, tipo: Text, valor: R, fecha: DateTime ) ]  
    <-- | pedirMediciones() --x  
|  
-----
```

La interfaz debe tener seguir este diseño:



1. Una sección “Mediciones almacenadas”, que será un gráfico y debe actualizarse automáticamente.
2. Un estado visible de carga, éxito y error.
3. Un diseño responsive, claro y accesible, sin depender de un framework visual externo.

El navegador debe usar exclusivamente `PeticionarioRest.pedirMediciones()`; no debe acceder directamente a la base de datos.